

ML Loop Code-Along - Step 12

Cross-validation and final recap

Incremental prerequisite prep for Imperial ML/AI coursework

Prepared for Anthony Watts - June 2026

Rule: nothing appears before it is explained.

12. Cross-validation: less faith in one lucky split

A single train/test split can be lucky or unlucky. Cross-validation repeats the train/evaluate idea across several folds and reports a set of scores.

Step 12.1: Import cross-validation scoring

Why this cell exists

`cross_val_score` evaluates a model across multiple splits. With a pipeline, preprocessing happens safely inside each fold.

```
from sklearn.model_selection import cross_val_score
```

What to look for

This is another reason pipelines matter: they help avoid leakage during repeated evaluation.

Step 12.2: Score the pipeline over five folds

Why this cell exists

`cv=5` means five train/evaluate rounds. `scoring="accuracy"` asks for accuracy in each round.

```
cv_scores = cross_val_score(
    knn_pipeline,
    X_cls,
    y_cls,
    cv=5,
    scoring="accuracy",
)

print("Scores:", cv_scores.round(3))
print("Mean:", round(cv_scores.mean(), 3))
print("Std:", round(cv_scores.std(), 3))
```

What to look for

The mean gives a rough central estimate. The standard deviation hints at how much scores varied across folds.

13. What you should now be able to say

- I can point to X and y in a supervised-learning problem.
- I can explain why splitting before fitting matters.
- I know that `fit()` learns from training data and `predict()` applies the learned model to feature rows.
- I can calculate a simple regression error and explain residuals as actual minus predicted.
- I can recognise overfitting as a train/test performance gap, not just as a scary word.
- I know why accuracy can be misleading for imbalanced classification problems.
- I know why KNN needs scaling and why scaling must be fitted on training data only.
- I know why Pipeline is the tidy, safer way to combine preprocessing and modelling.

The ML Loop, One Cell at a Time - generated June 2026

The one-sentence ML loop

We represent examples as features, train a model to map those features to known answers, measure how wrong its predictions are, adjust or choose models to reduce useful error, and evaluate on unseen data to estimate generalisation.

14. Small extension exercises

These are deliberately small. Do not turn them into a new side quest unless one of them exposes a real gap.

1. Change the house-price feature list. Remove `distance_station_km` and rerun. What changed?
2. In the diabetes regression example, compare `LinearRegression` with `DecisionTreeRegressor(max_depth=2, 3, 4, and None)`.
3. For classification, change `KNeighborsClassifier(n_neighbors=5)` to 1, 3, 7, and 15. What happens?
4. Change the classification metric from accuracy to recall in `cross_val_score`. Why might the conclusion change?
5. Add one markdown cell explaining the entire notebook in ten sentences or fewer.

15. Optional reinforcement, not required pre-reading

Use these only when a concept feels wobbly after doing the code. Do not watch everything before coding.

Concept	Best reinforcement	Why
Features, labels, loss, generalisation	Google Machine Learning Crash Course	Good conceptual overview of the basic supervised-learning loop.
fit/predict, train/test split, estimators	scikit-learn Getting Started and User Guide	Official source for the API you are actually using.
Scaling and leakage	scikit-learn Common Pitfalls and StandardScaler docs	Best source for why preprocessing belongs in a careful workflow.
Regression intuition	StatQuest linear regression videos	Plain-English reinforcement if residuals or coefficients feel slippery.
Classification metrics	Google Classification module or StatQuest confusion matrix	Good backup for precision, recall, F1, ROC/AUC, and confusion matrices.
Trees and ensembles	StatQuest decision trees, random forests, boosting	Useful after you have seen overfitting with a decision tree.

References used for chapter design

Google Machine Learning Crash Course: <https://developers.google.com/machine-learning/crash-course>

Google ML Crash Course, Classification: <https://developers.google.com/machine-learning/crash-course/classification>

scikit-learn Getting Started: https://scikit-learn.org/stable/getting_started.html

scikit-learn `train_test_split`: https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.train_test_split.html

scikit-learn `StandardScaler`: <https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.StandardScaler.html>

scikit-learn Common pitfalls and recommended practices: https://scikit-learn.org/stable/common_pitfalls.html

scikit-learn User Guide: https://scikit-learn.org/stable/user_guide.html

StatQuest video index: <https://statquest.org/video-index/>